SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03	**Course Name: Natural Language Processing**
Branch: CSE	**Year: 2026**
Name: Rakshitha V B	**Reg No: 192324004**
Date: 05/11/2026	**Dept: AI&DS**

Assignment 2

Question 1: Add-1 Smoothed Bigram Probabilities

Figure 4.1 gives bigram counts for eight words of the Berkeley Restaurant Project corpus (V = 1446 word types, 9332 sentences). Most cells are zero, so an unsmoothed model would assign zero probability to many unseen word pairs. Add-1 (Laplace) smoothing fixes this by adding one to every bigram count and adding V to the denominator.

Formulae

$$\text{Unsmoothed: } P(w_n \mid w_{n-1}) = C(w_{n-1}\, w_n) / C(w_{n-1})$$

$$\text{Add-1: } P^*(w_n \mid w_{n-1}) = (C(w_{n-1}\, w_n) + 1) / (C(w_{n-1}) + V)$$

$$\text{Reconstituted count: } c^*(w_{n-1}\, w_n) = (C(w_{n-1}\, w_n) + 1) \times C(w_{n-1}) / (C(w_{n-1}) + V)$$

The unigram counts needed for the denominators are taken from the same corpus: i = 2533, want = 927, to = 2417, eat = 746, chinese = 158, food = 1093, lunch = 341, spend = 278.

Program (q1_addone.py)

```
words = ["i", "want", "to", "eat", "chinese", "food", "lunch", "spend"]


counts = [
    [5, 827, 0, 9, 0, 0, 0, 2],
    [2, 0, 608, 1, 6, 6, 5, 1],
    [2, 0, 4, 686, 2, 0, 6, 211],
    [0, 0, 2, 0, 16, 2, 42, 0],
    [1, 0, 0, 0, 0, 82, 1, 0],
    [15, 0, 15, 0, 1, 4, 0, 0],
    [2, 0, 0, 0, 0, 1, 0, 0],
    [1, 0, 1, 0, 0, 0, 0, 0],
]


unigrams = [2533, 927, 2417, 746, 158, 1093, 341, 278]
V = 1446



def print_table(title, table, fmt):
    print(title)
    header = " " * 9 + "".join(f"{w:>9}" for w in words)
    print(header)
    for w, row in zip(words, table):
        print(f"{w:<9}" + "".join(f"{fmt.format(v):>9}" for v in row))
    print()
```

```
smoothed_counts = [[c + 1 for c in row] for row in counts]

smoothed_probs = [
    [(c + 1) / (unigrams[i] + V) for c in row]
    for i, row in enumerate(counts)
]

reconstituted = [
    [(c + 1) * unigrams[i] / (unigrams[i] + V) for c in row]
    for i, row in enumerate(counts)
]

print_table("Add-1 smoothed bigram counts C(w1 w2) + 1", smoothed_counts, "{:d}")
print_table("Add-1 smoothed bigram probabilities P*(w2 | w1)", smoothed_probs,
"{:.4f}")
print_table("Reconstituted counts c*(w1 w2)", reconstituted, "{:.1f}")

print("Sample calculations")
print(f"P*(want | i)    = (827 + 1) / (2533 + {V}) = {828 / (2533 + V):.4f}")
print(f"P*(chinese | i) = (0 + 1) / (2533 + {V}) = {1 / (2533 + V):.6f}")
print(f"P*(eat | to)    = (686 + 1) / (2417 + {V}) = {687 / (2417 + V):.4f}")
```

Output: add-1 smoothed bigram counts

```
Add-1 smoothed bigram counts C(w1 w2) + 1
                 i     want       to      eat  chinese     food    lunch    spend
i                6      828        1       10        1        1        1        3
want             3        1      609        2        7        7        6        2
to               3        1        5      687        3        1        7      212
eat              1        1        3        1       17        3       43        1
chinese          2        1        1        1        1       83        2        1
food            16        1       16        1        2        5        1        1
lunch            3        1        1        1        1        2        1        1
spend            2        1        2        1        1        1        1        1
```

Output: add-1 smoothed bigram probabilities

```
Add-1 smoothed bigram probabilities P*(w2 | w1)
                 i     want       to      eat  chinese     food    lunch    spend
i           0.0015   0.2081   0.0003   0.0025   0.0003   0.0003   0.0003   0.0008
want        0.0013   0.0004   0.2566   0.0008   0.0029   0.0029   0.0025   0.0008
to          0.0008   0.0003   0.0013   0.1778   0.0008   0.0003   0.0018   0.0549
eat         0.0005   0.0005   0.0014   0.0005   0.0078   0.0014   0.0196   0.0005
chinese     0.0012   0.0006   0.0006   0.0006   0.0006   0.0517   0.0012   0.0006
food        0.0063   0.0004   0.0063   0.0004   0.0008   0.0020   0.0004   0.0004
lunch       0.0017   0.0006   0.0006   0.0006   0.0006   0.0011   0.0006   0.0006
spend       0.0012   0.0006   0.0012   0.0006   0.0006   0.0006   0.0006   0.0006
```

Output: reconstituted counts

Reconstituted counts c*(w1 w2)

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527.1	0.6	6.4	0.6	0.6	0.6	1.9
want	1.2	0.4	237.9	0.8	2.7	2.7	2.3	0.8
to	1.9	0.6	3.1	429.8	1.9	0.6	4.4	132.6
eat	0.3	0.3	1.0	0.3	5.8	1.0	14.6	0.3
chinese	0.2	0.1	0.1	0.1	0.1	8.2	0.2	0.1
food	6.9	0.4	6.9	0.4	0.9	2.2	0.4	0.4
lunch	0.6	0.2	0.2	0.2	0.2	0.4	0.2	0.2
spend	0.3	0.2	0.3	0.2	0.2	0.2	0.2	0.2

Output: sample calculations

```
Sample calculations
P*(want | i)    = (827 + 1) / (2533 + 1446) = 0.2081
P*(chinese | i) = (0 + 1) / (2533 + 1446) = 0.000251
P*(eat | to)    = (686 + 1) / (2417 + 1446) = 0.1778
```

Observation

Smoothing moves probability mass from seen to unseen bigrams. P(want | i) drops from 827/2533 = 0.3265 to 0.2081, while a bigram never observed, such as (i, chinese), rises from 0 to 0.00025. The reconstituted counts show the same effect: c*(i want) falls from 827 to about 527, and c*(to eat) falls from 686 to about 430. This large discount on frequent bigrams is the well-known drawback of add-1 smoothing when V is large relative to the context counts.

Question 2: Bag Generation Using an N-gram Model

Bag generation takes a shuffled bag of words and tries to recover the original sentence order. The program trains a bigram language model with add-1 smoothing on a restaurant-domain corpus, using sentence start and end markers, and then searches for the word ordering with the highest probability.

Approach

1. The corpus is tokenised and bigram and unigram counts are collected, including the start marker and the end marker.

2. Each bigram probability is computed with add-1 smoothing, and log probabilities are summed to avoid numerical underflow.

3. A beam search builds the sentence left to right. At every step it appends each word still left in the bag, keeps only the best 50 partial orderings, and finally adds the end-marker probability to rank the complete sentences.

4. Because the search works on a multiset of words, repeated words such as the two occurrences of “the” are handled correctly.

Program (q2_bag_generation.py)

```
import math
import random
from collections import Counter


CORPUS = [
    "i want to eat chinese food",
    "i want to eat lunch",
    "i want to spend money on food",
    "i would like to eat british food",
    "tell me about chez panisse",
    "can you give me a listing of the kinds of food that are available",
    "i am looking for a good place to eat breakfast",
    "i want to eat dinner at a cheap restaurant",
    "i want chinese food",
    "give me a cheap chinese restaurant",
    "i need to find a place to eat lunch",
    "where is a good chinese restaurant",
    "i would like to spend less on lunch",
    "i want to find a restaurant that serves chinese food",
    "tell me where to eat lunch",
    "the restaurant serves good food",
    "she wants to eat lunch at the restaurant",
    "we want to eat dinner at a good restaurant",
    "he wants to spend money on a good lunch",
    "i am hungry and i want to eat food",
    "show me a cheap place to eat dinner",
    "i want to eat at a chinese restaurant",
    "what is a good place to eat lunch",
    "i have to eat lunch at noon",
    "the food at the restaurant is cheap and good",
]


START = "<s>"
END = "</s>"
```

```
class BigramModel:
    def __init__(self, sentences):
        self.unigrams = Counter()
        self.bigrams = Counter()
        for sentence in sentences:
            tokens = [START] + sentence.lower().split() + [END]
            self.unigrams.update(tokens)
            self.bigrams.update(zip(tokens, tokens[1:]))
        self.vocab_size = len(self.unigrams)

    def log_prob(self, prev, word):
        numerator = self.bigrams[(prev, word)] + 1
        denominator = self.unigrams[prev] + self.vocab_size
        return math.log(numerator / denominator)

    def score(self, words):
        tokens = [START] + list(words) + [END]
        return sum(self.log_prob(a, b) for a, b in zip(tokens, tokens[1:]))



def generate_order(model, bag, beam_width=50):
    bag = [w.lower() for w in bag]
    beam = [(0.0, [START], Counter(bag))]
    for _ in range(len(bag)):
        candidates = {}
        for score, sequence, remaining in beam:
            for word in remaining:
                if remaining[word] == 0:
                    continue
                new_score = score + model.log_prob(sequence[-1], word)
                new_remaining = remaining.copy()
                new_remaining[word] -= 1
                key = (tuple(sequence[1:] + [word]),)
                candidates[key] = (new_score, sequence + [word], new_remaining)
        beam = sorted(candidates.values(), key=lambda item: item[0],
reverse=True)[:beam_width]
    finished = []
    for score, sequence, remaining in beam:
        total = score + model.log_prob(sequence[-1], END)
        finished.append((total, sequence[1:]))
    finished.sort(key=lambda item: item[0], reverse=True)
    return finished



def main():
    model = BigramModel(CORPUS)
    random.seed(7)
    tests = [
        "i want to eat chinese food",
        "i would like to spend money on lunch",
        "tell me where to eat dinner",
        "give me a cheap chinese restaurant",
        "the food at the restaurant is good",
        "she wants to eat lunch at a restaurant",
    ]
    print(f"Corpus sentences : {len(CORPUS)}")
    print(f"Vocabulary size  : {model.vocab_size}")
    print("-" * 64)
    correct = 0
    for sentence in tests:
        bag = sentence.split()
        random.shuffle(bag)
        ranked = generate_order(model, bag)
        best_score, best = ranked[0]
        guess = " ".join(best)
```

```
        match = guess == sentence
        correct += match
        print(f"Bag of words : {bag}")
        print(f"Best guess   : {guess}")
        print(f"Original     : {sentence}")
        print(f"Log prob     : {best_score:.3f}")
        print(f"Exact match  : {'YES' if match else 'NO'}")
        print("-" * 64)
    print(f"Exact matches: {correct} / {len(tests)}")



if __name__ == "__main__":
    main()
```

Output

```
Corpus sentences : 25
Vocabulary size  : 57
----------------------------------------------------------------
Bag of words : ['chinese', 'i', 'food', 'eat', 'want', 'to']
Best guess   : i want to eat chinese food
Original     : i want to eat chinese food
Log prob     : -15.885
Exact match  : YES
----------------------------------------------------------------
Bag of words : ['lunch', 'money', 'to', 'like', 'i', 'spend', 'on', 'would']
Best guess   : i would like to spend money on lunch
Original     : i would like to spend money on lunch
Log prob     : -25.438
Exact match  : YES
----------------------------------------------------------------
Bag of words : ['where', 'dinner', 'to', 'tell', 'me', 'eat']
Best guess   : tell me where to eat dinner
Original     : tell me where to eat dinner
Log prob     : -21.006
Exact match  : YES
----------------------------------------------------------------
Bag of words : ['a', 'restaurant', 'chinese', 'me', 'give', 'cheap']
Best guess   : give me a cheap chinese restaurant
Original     : give me a cheap chinese restaurant
```

```
Log prob     : -20.699
Exact match  : YES
------------------------------------------------------------
Bag of words : ['the', 'is', 'at', 'food', 'good', 'restaurant', 'the']
Best guess   : the restaurant is at the good food
Original     : the food at the restaurant is good
Log prob     : -26.387
Exact match  : NO
------------------------------------------------------------
Bag of words : ['lunch', 'to', 'eat', 'at', 'restaurant', 'wants', 'she', 'a']
Best guess   : she wants to eat lunch at a restaurant
Original     : she wants to eat lunch at a restaurant
Log prob     : -25.617
Exact match  : YES
------------------------------------------------------------
Exact matches: 5 / 6
```

Observation

The model recovered 5 of the 6 sentences exactly. The failure, “the restaurant is at the good food”, is still locally fluent, which shows the limit of a bigram model: it only sees one word of context and cannot tell that “the food at the restaurant is good” is more natural. A larger corpus or a trigram model would improve accuracy.

Question 3: Penn Treebank Tagging

Each word of the sentences from Damon Runyon's short stories is tagged with the Penn Treebank tagset. Punctuation is ignored, and the possessive 's in sentence (e) is split from its noun as the Treebank convention requires.

(a) It is a nice night.

It/PRP is/VBZ a/DT nice/JJ night/NN

No.	Word	Tag	Meaning
1	It	**PRP**	Personal pronoun
2	is	**VBZ**	Verb, 3rd person singular present
3	a	**DT**	Determiner
4	nice	**JJ**	Adjective
5	night	**NN**	Noun, singular or mass

(b) This crap game is over a garage in Fifty-second Street.

This/DT crap/NN game/NN is/VBZ over/IN a/DT garage/NN in/IN Fifty-second/JJ Street/NNP

No.	Word	Tag	Meaning
1	This	**DT**	Determiner
2	crap	**NN**	Noun, singular or mass
3	game	**NN**	Noun, singular or mass
4	is	**VBZ**	Verb, 3rd person singular present
5	over	**IN**	Preposition / subordinating conjunction
6	a	**DT**	Determiner
7	garage	**NN**	Noun, singular or mass
8	in	**IN**	Preposition / subordinating conjunction
9	Fifty-second	**JJ**	Adjective
10	Street	**NNP**	Proper noun, singular

(c) Nobody ever takes the newspapers she sells.

Nobody/NN ever/RB takes/VBZ the/DT newspapers/NNS she/PRP sells/VBZ

No.	Word	Tag	Meaning
1	Nobody	**NN**	Noun, singular or mass
2	ever	**RB**	Adverb
3	takes	**VBZ**	Verb, 3rd person singular present
4	the	**DT**	Determiner
5	newspapers	**NNS**	Noun, plural
6	she	**PRP**	Personal pronoun
7	sells	**VBZ**	Verb, 3rd person singular present

(d) He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.

He/PRP is/VBZ a/DT tall/JJ skinny/JJ guy/NN with/IN a/DT long/JJ sad/JJ mean-looking/JJ kisser/NN and/CC a/DT mournful/JJ voice/NN

No.	Word	Tag	Meaning
1	He	**PRP**	Personal pronoun
2	is	**VBZ**	Verb, 3rd person singular present
3	a	**DT**	Determiner
4	tall	**JJ**	Adjective
5	skinny	**JJ**	Adjective
6	guy	**NN**	Noun, singular or mass
7	with	**IN**	Preposition / subordinating conjunction
8	a	**DT**	Determiner
9	long	**JJ**	Adjective
10	sad	**JJ**	Adjective
11	mean-looking	**JJ**	Adjective
12	kisser	**NN**	Noun, singular or mass
13	and	**CC**	Coordinating conjunction
14	a	**DT**	Determiner
15	mournful	**JJ**	Adjective
16	voice	**NN**	Noun, singular or mass

(e) I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of.

I/PRP am/VBP sitting/VBG in/IN Mindy/NNP 's/POS restaurant/NN putting/VBG on/RP the/DT gefillte/JJ fish/NN which/WDT is/VBZ a/DT dish/NN I/PRP am/VBP very/RB fond/JJ of/IN

No.	Word	Tag	Meaning
1	I	**PRP**	Personal pronoun
2	am	**VBP**	Verb, non-3rd person singular present
3	sitting	**VBG**	Verb, gerund / present participle
4	in	**IN**	Preposition / subordinating conjunction
5	Mindy	**NNP**	Proper noun, singular
6	’s	**POS**	Possessive ending
7	restaurant	**NN**	Noun, singular or mass
8	putting	**VBG**	Verb, gerund / present participle
9	on	**RP**	Particle
10	the	**DT**	Determiner
11	gefillte	**JJ**	Adjective
12	fish	**NN**	Noun, singular or mass
13	which	**WDT**	Wh-determiner
14	is	**VBZ**	Verb, 3rd person singular present
15	a	**DT**	Determiner
16	dish	**NN**	Noun, singular or mass
17	I	**PRP**	Personal pronoun
18	am	**VBP**	Verb, non-3rd person singular present
19	very	**RB**	Adverb
20	fond	**JJ**	Adjective
21	of	**IN**	Preposition / subordinating conjunction

(f) When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.

When/WRB a/DT guy/NN and/CC a/DT doll/NN get/VBP to/TO taking/VBG peeks/NNS back/RB and/CC forth/RB at/IN each/DT other/JJ why/WRB there/RB you/PRP are/VBP indeed/RB

No.	Word	Tag	Meaning
1	When	**WRB**	Wh-adverb
2	a	**DT**	Determiner
3	guy	**NN**	Noun, singular or mass
4	and	**CC**	Coordinating conjunction
5	a	**DT**	Determiner

No.	Word	Tag	Meaning
6	doll	**NN**	Noun, singular or mass
7	get	**VBP**	Verb, non-3rd person singular present
8	to	**TO**	to
9	taking	**VBG**	Verb, gerund / present participle
10	peeks	**NNS**	Noun, plural
11	back	**RB**	Adverb
12	and	**CC**	Coordinating conjunction
13	forth	**RB**	Adverb
14	at	**IN**	Preposition / subordinating conjunction
15	each	**DT**	Determiner
16	other	**JJ**	Adjective
17	why	**WRB**	Wh-adverb
18	there	**RB**	Adverb
19	you	**PRP**	Personal pronoun
20	are	**VBP**	Verb, non-3rd person singular present
21	indeed	**RB**	Adverb

Notes on the difficult cases

“Nobody” is tagged NN, as the Treebank treats indefinite pronouns of this kind as nouns. “Fifty-second” is an adjective modifying the proper noun “Street”. “on” in “putting on the gefillte fish” is a particle (RP) because it belongs to the phrasal verb. “each other” is tagged DT followed by JJ, following Treebank practice. In (f), “back” and “forth” are adverbs, “why” is a wh-adverb used as an interjection-like opener, and “there” is a locative adverb.